

ADR-06: Deudas técnicas documentadas de ZenFlow

ZenFlow — Reconocimiento formal de decisiones que generan costo técnico futuro

Campo	Valor
Autor	Dylan Vazquez
Fecha	15/07/2026
Estado	Propuesto

Contexto

Los ADR-01 al ADR-05 documentaron las decisiones arquitectónicas de ZenFlow: tecnología, persistencia, estilo arquitectónico, API REST y patrones GOF. Cada uno de esos ADRs ya señaló consecuencias negativas — limitaciones técnicas y deudas — como parte de la sección de consecuencias. Este ADR las formaliza por separado porque merecen su propio espacio de análisis.

Una deuda técnica no es un error — es una decisión consciente (o a veces un descuido) que resuelve un problema de corto plazo a cambio de un costo futuro. El problema no es tenerlas, sino no documentarlas. Si no están escritas, nadie sabe que existen hasta que algo se rompe.

ZenFlow tiene dos deudas técnicas concretas que se identificaron durante el desarrollo y que este ADR documenta formalmente con su propuesta de solución.

Deuda Técnica 1 — Token de autenticación hardcodeado en el código fuente

¿Qué es?

En `TareaServiceProxy.cs`` y `HabitoServiceProxy.cs`` el token de autenticación de la API está escrito directamente en el código:

```
private readonly string _tokenEsperado = "zenflow-token-2025";
```

Esto significa que el token vive en el repositorio de GitHub, visible para cualquier persona que tenga acceso al código. No es un placeholder ni un ejemplo — es el token real que protege los endpoints de la API.

Esta deuda es de categoría infraestructura/configuración: un parámetro de seguridad que debería vivir fuera del código fuente está embebido directamente en él.

¿Por qué existe?

Existe por una decisión consciente de velocidad. El ADR-05 ya documentó que 'el token estático hardcodeado en TareaServiceProxy es un punto de partida funcional, pero no es seguro para exposición pública'. Se eligió así para tener autenticación básica funcionando durante el semestre sin invertir tiempo en configurar un sistema de variables de entorno o JWT, que tienen su propia curva de aprendizaje. La fecha de entrega del proyecto fue el factor determinante.

Costo de no pagarla

- Cualquier persona con acceso al repositorio de GitHub conoce el token y puede hacer llamadas a la API sin restricción — leer, crear y eliminar tareas y hábitos.
- Si el proyecto crece y la API se expone en una red pública, el token hardcodeado se convierte en una vulnerabilidad de seguridad real, no solo teórica.
- Cambiar el token requiere modificar el código fuente, recompilar y redespargar — en lugar de simplemente cambiar una variable de entorno. Eso hace que la rotación de credenciales sea costosa y propensa a errores.
- Si hay más de un entorno (desarrollo, producción), ambos usan el mismo token porque está hardcodeado, lo que elimina cualquier separación de ambientes.

Propuesta de solución

La técnica de refactorización aplicable es Extract Variable hacia una fuente de configuración externa. En lugar de leer el token del código, leerlo de una variable de entorno o de un archivo de configuración excluido del repositorio:

```
// Antes — hardcodeado en el código
private readonly string _tokenEsperado = "zenflow-token-2025";
```

```
// Después — leído de variable de entorno
private readonly string _tokenEsperado =
    Environment.GetEnvironmentVariable("ZENFLOW_API_TOKEN")
    ?? throw new InvalidOperationException("Token no configurado.");
```

El archivo `.env` o la configuración de variables del sistema nunca se sube al repositorio — se agrega al `.gitignore`. Cada desarrollador o servidor configura su propio valor. Cuando el proyecto madure, se puede migrar a JWT sin cambiar la arquitectura — solo cambia cómo se valida el token dentro del Proxy.

Resumen — Deuda Técnica 1

Categoría: Infraestructura / Configuración

Origen: Decisión consciente por tiempo limitado en el semestre

Riesgo: Credencial de seguridad expuesta en repositorio público

Solución: Variables de entorno con `Environment.GetEnvironmentVariable()`

Deuda Técnica 2 — Persistencia en JSON sin soporte de transacciones

¿Qué es?

ZenFlow guarda todos sus datos en archivos JSON planos: ``tareas.json``, ``habitos.json`` y ``apps.json``. El mecanismo de escritura es simple: leer el archivo completo, modificar la lista en memoria y sobrescribir el archivo completo. No hay transacciones, no hay rollback y no hay validación de integridad.

Esto significa que si la aplicación se cierra inesperadamente — por un crash, por falta de luz o por forzar el cierre — en medio de una operación de escritura, el archivo JSON puede quedar corrupto o incompleto. Cuando ZenFlow intente leerlo la próxima vez, el ``JsonSerializer.Deserialize`` lanzará una excepción y la app no podrá iniciar.

¿Por qué existe?

Existe porque JSON fue la decisión correcta para la etapa inicial del proyecto. El ADR-02 justificó explícitamente empezar con JSON: permite iterar rápido, los archivos son legibles por humanos y no requieren configuración adicional. La deuda no es que se haya elegido JSON — la deuda es que no se implementó ningún mecanismo de protección contra escrituras parciales, lo que es un descuido que no se detectó a tiempo durante el desarrollo.

Costo de no pagarla

- Una escritura interrumpida puede corromper el archivo JSON y dejar a ZenFlow en un estado de inicio imposible — el usuario pierde todos sus datos sin posibilidad de recuperación automática.
- No hay forma de deshacer una operación incorrecta. Si el usuario elimina accidentalmente una tarea, no hay rollback — el archivo ya fue sobrescrito.
- A medida que el volumen de datos crece, leer y sobrescribir el archivo completo en cada operación se vuelve cada vez más lento. Con 10 tareas es imperceptible; con 10,000 es un problema real de rendimiento.
- Si la API y la app WPF intentan escribir simultáneamente, el Singleton con lock serializa las operaciones pero no garantiza atomicidad — una escritura puede quedar a medias si el proceso es interrumpido durante el ``File.WriteAllText``.

Propuesta de solución

Hay dos niveles de solución dependiendo del tiempo disponible:

Nivel 1 — Escritura atómica con archivo temporal (corto plazo):

En lugar de sobrescribir el archivo directamente, escribir en un archivo temporal y luego renombrarlo. El sistema operativo garantiza que el renombrado es atómico — o el archivo viejo existe completo o el nuevo existe completo, nunca un estado intermedio corrupto:

```
// En lugar de File.WriteAllText(_ruta, json):  
var rutaTmp = _ruta + ".tmp";  
File.WriteAllText(rutaTmp, json);  
File.Move(rutaTmp, _ruta, overwrite: true); // atómico
```

Nivel 2 — Migración a SQLite (largo plazo):

SQLite tiene soporte nativo de transacciones ACID. Gracias al DIP implementado en ADR-02, la migración no requiere tocar GestorTareas ni la UI — solo se crea TareaRepoSQLite que implementa ITareaRepo y se cambia una línea en App.xaml.cs. Esta es la solución definitiva documentada desde el inicio del proyecto.

Resumen — Deuda Técnica 2

Categoría: Persistencia / Integridad de datos

Origen: Descuido no detectado durante el desarrollo inicial

Riesgo: Corrupción de archivos JSON ante escrituras interrumpidas

Solución corto plazo: Escritura atómica con archivo temporal + File.Move

Solución largo plazo: Migración a SQLite via TareaRepoSQLite (DIP ya preparado)

Comparativa de deudas

Categoría	Infraestructura / Seguridad	Persistencia / Integridad
Origen	Decisión consciente por tiempo	Descuido no detectado
Probabilidad de impacto	Alta si el repo es público	Media — depende de crashes
Costo de solución	Bajo — una tarde de trabajo	Medio — migración a SQLite
Urgencia	Alta	Media

La Deuda 1 tiene mayor urgencia porque afecta la seguridad del sistema desde el momento en que el repositorio es público. La Deuda 2 tiene menor probabilidad de impacto inmediato pero mayor costo si no se paga antes de que el volumen de datos crezca o la app se use en condiciones de inestabilidad.

Declaración de uso de IA

Este ADR fue desarrollado con el apoyo de Claude como herramienta de consulta y estructuración. Las deudas técnicas documentadas son reales y surgieron del proceso de desarrollo de ZenFlow durante el semestre. Claude contribuyó a formalizar el análisis y la propuesta de solución de cada deuda con el rigor que un ADR requiere.